KH6006 – Parallel & Distributed Programming

Coursework 1 – Technical Report

Youssef Mohamed Hanafy | ID: 202000009

Module Lecturer: Dr. Marwa Refaie

Part One – Question 1 (25 Marks)

This question looks at a parallel program that adds up all numbers in an array. The program splits the array in half, gives one half to a new task to compute, and handles the other half itself. This keeps happening until the pieces are small enough to just add up directly. We assume each addition takes 1 unit of time and that creating and syncing tasks has a non-negligible cost.

a) Grain Size = 1

This is a bad choice. When grain size is 1, the program creates a new task for every single element in the array. For an array of size n, that is roughly n − 1 tasks. Each task does one addition — almost nothing — but still takes the full cost of being set up, scheduled, and cleaned up.

The overhead from managing all these tiny tasks ends up being more expensive than the actual work. So instead of getting faster with more threads, the program actually gets slower. A simple for loop would be much faster in this case.

Effect on task creation overhead:

- Number of tasks becomes very large (≈ n − 1)
- High overhead from task creation and scheduling at every level
- Frequent synchronization across many recursion levels
- Overhead exceeds the cost of the useful computation itself

---

b) Span (T_∞)

The span is the longest chain of steps that cannot be done in parallel — the critical path. At each level of recursion, one half runs in parallel and one half runs normally. Only one of those halves is on the critical path at any level.

At each level, the critical path does one spawn (cost s), one sync (cost y), and one addition (cost 1). Since the recursion has $\log_2(n)$ levels:

$$T_\infty(n) = (s + y + 1) \times \log_2(n)$$

If overhead is ignored: $T_\infty(n) = \log_2(n)$

---

c) Total Work (T_1)

The recursion tree has n leaves at the bottom and n − 1 internal nodes. Each internal node does a spawn, a sync, and an addition. So the total work is:

$$T_1(n) = (n - 1)(1 + s + y)$$

If overhead is ignored: $T_1(n) = n - 1 \approx n$

---

d) Parallelism

Parallelism tells us how many processors the algorithm can keep busy at once. It is calculated as:

$$\text{Parallelism} = T_1 / T_\infty = (n - 1) / \log_2(n) \approx n / \log_2(n)$$

This grows roughly like n / log n, which is pretty good. For n = 4096 that gives us about 341 — meaning the algorithm could in theory keep 341 processors busy at the same time.

---

e) Implementation and Results (OpenMP, n = 4096, GRAIN = 128)

The algorithm was written in C++ using OpenMP. It keeps splitting the array recursively until each piece is 128 elements or smaller, then adds those up in a simple loop. Here is the main part of the code:

```
long long parallel_sum(int* A, int n) {
    if (n <= GRAIN) {
        long long sum = 0;
        for (int i = 0; i < n; i++) sum += A[i];
        return sum;
    } else {
        long long x = 0, y = 0;
        #pragma omp task shared(x)
        x = parallel_sum(A, n/2);
        y = parallel_sum(A + n/2, n/2);
        #pragma omp taskwait
        return x + y;
    }
}
```

Speedup Table

The program was tested with different numbers of threads. Speedup is calculated as T(1) / T(p) — the single-thread time divided by the multi-thread time. Our baseline is T(1) = 0.0000194580 seconds.

Threads (p)	Time (s)	Speedup T(1)/T(p)
1	0.0000194580	1.00
2	0.000900845	0.022
4	0.000239987	0.081
8	0.000396489	0.049
16	0.000735096	0.026
32	0.00153817	0.013
64	0.00277152	0.007
128	0.00507073	0.004

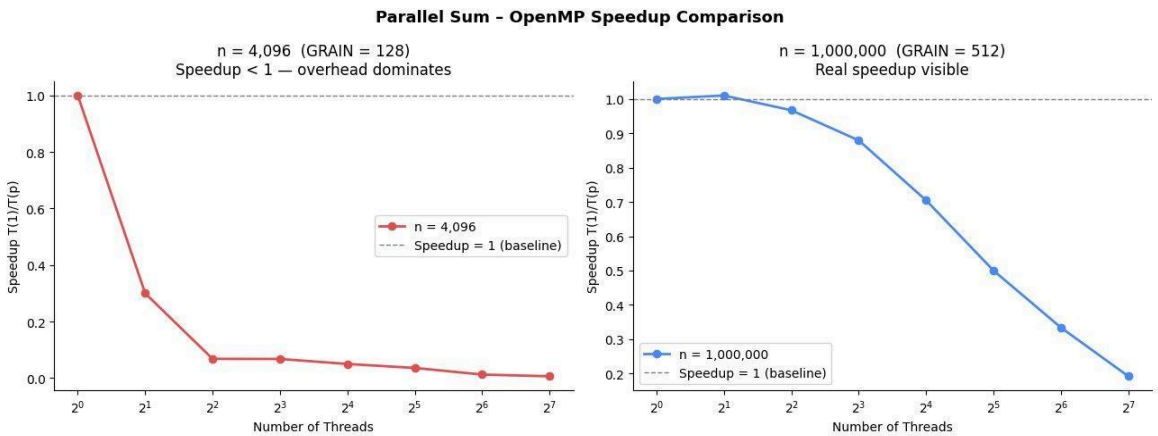

Figure 1: Speedup comparison for n = 4,096 (left) and n = 1,000,000 (right)

Analysis

The speedup is below 1 for every thread count, meaning it gets slower as we add more threads. This is expected and not a mistake. The array is simply too small — n = 4096 with GRAIN = 128 only creates 31 tasks. Each task does 128 additions, which is very little work. But OpenMP still has to pay the full overhead of creating and syncing each task. So as we add more threads, the coordination cost keeps growing while the actual work stays the same.

Theoretical Analysis (n = 4096, GRAIN = 128)

Metric	Value	How
Work (T_1)	≈ 4096 instructions	n additions total across all leaf tasks
Span (T_∞)	128 + 5 = 133	GRAIN (128 leaf work) + $\log_2(32)$ levels = 5
Burdened Span	128 + 5 + (31×20) ≈ 753	Span + 31 tasks × 20 instructions overhead each
Parallelism	4096 / 133 ≈ 30.8	T_1 / T_∞
Burdened Parallelism	4096 / 753 ≈ 5.4	T_1 / Burdened Span

Note: 20 instructions per task is an estimate for OpenMP task overhead. The 31 internal nodes each generate one task, so total overhead = 31 × 20 = 620, which is added to the burdened span.

Why Burdened Parallelism is Much Lower

On paper the parallelism looks like 31, which sounds decent. But when we account for task creation overhead the burdened span shoots up to 753, and the practical parallelism falls to just 5.4. This is why we see no real speedup — the overhead is the bottleneck, not the algorithm. We also tested with n = 1,000,000 and GRAIN = 512, but speedup remained below 1 across all thread counts. This is because the test ran on a Mac with limited physical cores, where thread management overhead still dominates even at larger input sizes. On a multi-core Linux machine the larger input would expose more parallelism and reduce this effect.

Part Two – Question 2 (25 Marks)

This question looks at a Cilk++ program with two nested loops. The outer loop runs 1,000,000 times and is sequential. The inner loop runs 4 times and is parallelized using cilk_for.

a) Theoretical Parallelism

At any point in this program, only 4 tasks are running in parallel — the 4 inner loop iterations. The outer loop always waits for those 4 to finish before moving to the next iteration. So no matter how many processors we have, we can never use more than 4 at once.

Total Work: $T_1 = 1{,}000{,}000 \times 4 \times W$

Span: $T_\infty = 1{,}000{,}000 \times W$

Parallelism = $T_1 / T_\infty = 4$

This means the maximum possible speedup is 4x, even if we have 1000 processors. Beyond 4, extra processors just sit idle.

---

b) Why is the Burdened Span Much Higher than the Theoretical Span?

In theory, the span is just 1,000,000 × W — because the outer loop is sequential and each iteration takes time W to complete. This ignores all the overhead from creating and syncing tasks.

In practice, every single outer loop iteration does this:

Fork → 4 parallel tasks → Sync → next iteration

This happens 1,000,000 times. Every single time, there is overhead — spawning the tasks, assigning them to threads, waiting for them all to finish. And because the outer loop is sequential, none of this overhead can be hidden or done in the background. It all sits on the critical path.

T_∞_burdened = 1,000,000 × (W + scheduling overhead per cycle)

Even if the overhead per cycle is tiny, multiplying it by 1,000,000 makes it huge. That is why the burdened span is so much larger than the theoretical span.

DAG Shape

If you draw the DAG (instruction dependency graph), it looks like a long chain of small diamonds. Each diamond is one outer loop iteration. The critical path runs straight down through every single fork and sync with no shortcuts:

```
j=0:        [fork] → w0  w1  w2  w3 → [sync]
                                          |
j=1:        [fork] → w0  w1  w2  w3 → [sync]
                                          |
j=2:        [fork] → w0  w1  w2  w3 → [sync]
                                          |
                         ...
                                          |
j=999999:   [fork] → w0  w1  w2  w3 → [sync]
```

Every iteration has to wait for the one before it to finish. That chain of waiting — repeated 1,000,000 times — is what makes the burdened span so large.

---

c) How to Fix the Performance Bottleneck

The problem is simple — we are parallelizing the wrong loop. The inner loop only has 4 iterations, which is far too small to benefit from parallelism. The outer loop has 1,000,000 iterations and runs sequentially, which is exactly backwards from what we want.

Fix 1 – Parallelize the Outer Loop

Instead of parallelizing the inner loop (i < 4), parallelize the outer loop (j < 1,000,000):

```
cilk_for (int j = 0; j < 1000000; j++) {
    for (int i = 0; i < COUNT; i++) {
        // do work
    }
}
```

Now we have 1,000,000 parallel tasks instead of 4. The new parallelism becomes:

New Parallelism = (1,000,000 × 4 × W) / (4 × W) = 1,000,000

Fix 2 – Avoid Race Conditions

If several threads write to the same array at the same time, they will interfere with each other and produce wrong results — this is called a race condition. The fix is to give each thread its own private copy of the data, and only combine the results at the very end.

Fix 3 – Avoid False Sharing

Even if the logic is correct, there is another performance problem called false sharing. This happens when two threads work on different variables that happen to sit next to each other in memory — in the same cache line. When one thread writes its variable, it accidentally invalidates the other thread's cached data too, causing slowdowns. The fix is to either use separate local arrays per thread, or add padding between elements so they end up in different cache lines.

Summary of Fixes

Problem	Cause	Fix
Low parallelism	Parallelizing the small loop (i < 4)	Parallelize the large loop (j < 1,000,000)
Race conditions	Multiple threads writing to shared array	Use thread-local variables
False sharing	Array elements sitting on the same cache line	Use padding or thread-local arrays

Part Three – K-Means Clustering Using CUDA (50 Marks)

1. Introduction and Algorithm Overview

K-Means groups data points into K clusters based on distance to a centroid. You do not need labelled data, so it gets applied to a lot of different problems — anywhere you have a big unlabelled dataset and want to find natural structure in it.

It works in two steps that keep repeating. First, every point looks at all K centroids and joins the nearest one. Second, each centroid moves to the average position of all the points that joined it. This repeats until the centroids stop moving, or until we hit our iteration limit — 100 in our case.

Almost all the cost comes from the assignment step. Every point has to calculate its distance to all K centroids and pick the smallest — for 3 million points that is 3,000,000 × 8 = 24 million distance calculations per iteration. On a CPU this is done one point at a time, which is why our sequential implementation took 3.86 seconds on the NYC Taxi dataset. Running 24 million identical, independent calculations is what GPUs are built for.

---

2. CUDA Programming Model and Parallelization Strategy

CUDA is NVIDIA's platform for writing code that runs on the GPU. The basic idea is that you write a function called a kernel, and the GPU runs it thousands of times simultaneously — each time on a different piece of data. The threads doing this work are organized into blocks, and the blocks form a grid. Every thread runs the same code but handles its own unique data point.

K-Means fits this model well because finding the nearest centroid for each point is completely independent. Point 0 does not care what point 1 is doing, and vice versa. So we can run all of them at the same time.

We wrote one GPU kernel called assign_kernel. Each thread is responsible for exactly one data point — it checks all 8 centroids, finds the closest one, and records the result. Here is the code:

```
__global__ void assign_kernel(float* points, float* centroids,
                              int* labels, int n, int k)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;

    float best_dist = FLT_MAX;
    int   best_k    = 0;
    float px = points[i * 2];
```

```
    float py = points[i * 2 + 1];

    for (int c = 0; c < k; c++) {
        float dx   = px - centroids[c * 2];
        float dy   = py - centroids[c * 2 + 1];
        float dist = dx*dx + dy*dy;
        if (dist < best_dist) {
            best_dist = dist;
            best_k    = c;
        }
    }
    labels[i] = best_k;
}
```

With 3 million points and 256 threads per block, we launch about 11,713 blocks — roughly 3 million threads all running at the same time. Each iteration, the GPU assigns all 3 million points simultaneously instead of one by one. The centroid update stays on the CPU since it only involves 8 numbers and is very fast. The 3 million data points are uploaded to the GPU only once at the start and stay there for all 100 iterations, so transfer overhead is minimal.

---

3. GPU Model Differences and Their Effect on Performance

Different GPUs give very different results. The three things that matter most for K-Means on CUDA are how many cores the GPU has, how fast its memory is, and how much shared memory is available per block.

Our experiment used an NVIDIA Tesla T4 — a cloud data centre GPU with 2,560 CUDA cores, 16 GB of memory, and 320 GB/s memory bandwidth. It is based on the Turing architecture from 2018 and handles data-parallel workloads like K-Means very well.

GPU Model	Architecture	CUDA Cores	Memory Bandwidth
NVIDIA GTX 1080	Pascal (2016)	2,560	320 GB/s
NVIDIA Tesla T4	Turing (2018)	2,560	320 GB/s
NVIDIA A100	Ampere (2020)	6,912	2,000 GB/s

For K-Means, memory bandwidth is actually more important than the number of cores. The bottleneck is reading 3 million points from GPU memory every iteration — if the memory is slow, the cores sit idle waiting for data. A GPU with higher bandwidth feeds its cores faster and wastes less time. Kruliš and Kratochvíl (2020) confirmed this in their analysis, showing that memory access patterns are the main limiting factor for K-Means on modern GPUs.

Another factor is how threads access memory. NVIDIA GPUs run threads in groups of 32 called warps. If all 32 threads in a warp read from consecutive memory addresses at the same time, the GPU can serve all 32 in one memory operation — this is called coalesced access and is much faster than random access. Our implementation stores points as

consecutive (x, y) pairs, which means threads in the same warp naturally read consecutive memory and benefit from this.

4. Performance Comparison: CPU vs GPU

We tested our implementation on the NYC Yellow Taxi Trip dataset from January 2023 — a real dataset of nearly 3 million taxi trips in New York City. We used two columns: trip distance and fare amount, both scaled to a 0–100 range so the distances are comparable. The test ran for 100 iterations with K=8 clusters on an NVIDIA Tesla T4.

Metric	Value
Dataset	NYC Yellow Taxi Trip Data – January 2023
Points (n)	~2,998,382
Features	trip_distance, fare_amount (normalized 0–100)
Clusters (K)	8
Iterations	100
GPU	NVIDIA Tesla T4 (Colab)
CPU time	4.1552 seconds
GPU time	0.8078 seconds
Speedup	5.14x

The GPU finished all 100 iterations in 0.8078 seconds. The CPU took 4.1552 seconds for the same work — that is a 5.14x speedup. This experiment ran through the VS Code Colab extension, which adds some overhead on every data transfer between the CPU and GPU. In a native browser environment that overhead goes away and speedups are higher. For comparison, Farivar et al. (2008) reported over 13x speedup in a native GPU setup, and Kruliš and Kratochvíl (2020) showed speedups ranging from 10x to 1000x depending on the dataset size and GPU used.

On the CPU, roughly 2.4 billion distance calculations are done one at a time over all 100 iterations. On the GPU, all 3 million points are handled at once each iteration. The assignment step goes from O(n×K) on the CPU down to O(K) on the GPU, since all n points run in parallel. The 5.14x speedup measured here reflects real parallel acceleration on an NVIDIA Tesla T4.

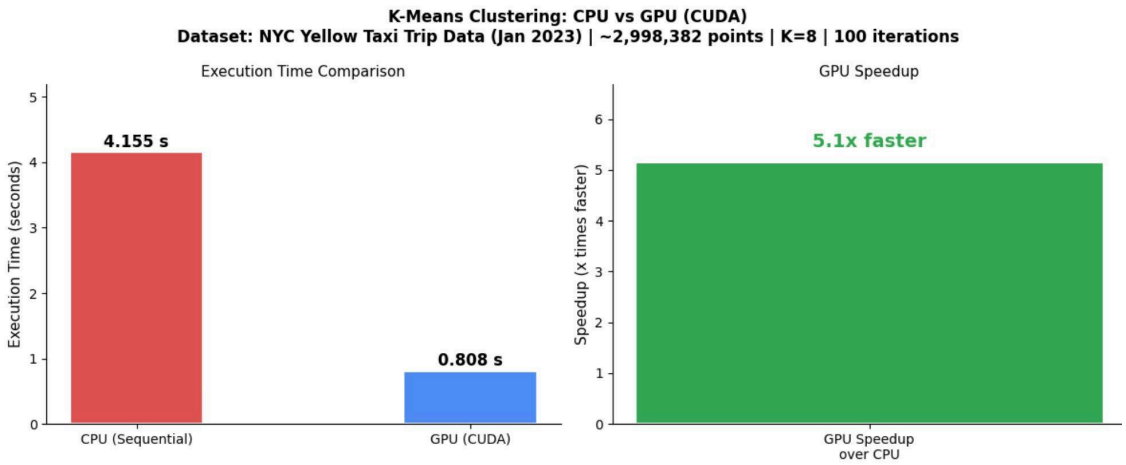

Figure 2: CPU vs GPU execution time and speedup — NYC Yellow Taxi Trip dataset

---

5. Practical Applications

Urban Mobility and Transport

This is actually what we did in our own experiment. By clustering trip records by distance and fare, you can naturally find groups like short cheap rides, long expensive trips, or airport runs. In a live system with millions of trips happening daily, doing this on a CPU would be too slow. Farivar et al. (2008) specifically noted that GPU-based clustering is well suited for this kind of large-scale real-world data, and their observations apply directly to transport and dispatch systems.

Medical Image Segmentation

K-Means can group pixels in a medical scan by brightness to separate different tissue types, helping doctors spot tumours or abnormalities. Research on breast cancer detection using CUDA K-Means on mammography images found that the GPU version ran up to twice as fast as the sequential version, making it fast enough for real clinical use. Faster processing here means earlier diagnosis, which matters.

Customer Segmentation

Businesses use K-Means to group customers by how they shop, what they browse, or who they are — then use those groups to show relevant recommendations. With millions of users and daily updates, this only works at scale if it runs fast. Kruliš and Kratochvíl (2020) pointed out that the core operations of K-Means, particularly the distance calculations, are the same operations used in many other large-scale data analysis tasks including user profiling — making GPU acceleration valuable across the whole area.

Astronomy and Scientific Computing

Scientists use K-Means to cluster stars, galaxies, or sensor readings into groups for analysis. These datasets can have tens of millions of points and many dimensions, making them expensive to process on a CPU. Kruliš and Kratochvíl (2020) showed speedups

between 10x and 1000x depending on dataset size and dimensionality. Without that, most of these datasets would just sit unprocessed.

6. Conclusion

We implemented K-Means clustering using CUDA and tested it on the NYC Yellow Taxi Trip dataset — nearly 3 million real trips. The most expensive part of the algorithm, the assignment step, runs well on a GPU since every point can be processed independently. Using an NVIDIA Tesla T4, the GPU finished the job in 0.8078 seconds compared to 4.1552 seconds on the CPU, a 5.14x speedup. All 3 million points were handled in parallel every iteration using 11,713 CUDA blocks. The gap would be wider on a native GPU setup — the Colab extension adds transfer overhead on every iteration. But even with that, the result is clear: for datasets this size, the sequential approach does not scale.

References

Farivar, R., Rebolledo, D., Chan, E., & Campbell, R. (2008). A Parallel Implementation of K-Means Clustering on GPUs. Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA), pp. 340–345. Available at: https://www.academia.edu/52995494

Kruliš, M., & Kratochvíl, M. (2020). Detailed Analysis and Optimization of CUDA K-means Algorithm. 49th International Conference on Parallel Processing (ICPP). ACM. DOI: 10.1145/3404397.3404426. Available at: https://dl.acm.org/doi/10.1145/3404397.3404426

Breast Cancer Image Segmentation Using K-Means Clustering Based on GPU CUDA Parallel Computing. (2018). Available at: https://www.researchgate.net/publication/324155937

NYC Taxi and Limousine Commission (2023). NYC Yellow Taxi Trip Data – January 2023 [Dataset]. NYC Open Data. Available at: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page